

# Module 23: Common Table Expressions (CTEs)

A Common Table Expression (CTE) is a temporary named result set that exists only during the execution of a query. CTEs make complex SQL queries easier to read, write, and maintain.



## Introduction

Consider the following query:

```
SELECT *
FROM
(
    SELECT DepartmentID,
           AVG(Salary) AS AvgSalary
    FROM Employees
    GROUP BY DepartmentID
) AS DepartmentAverage
WHERE AvgSalary > 50000;
```

This works, but becomes difficult to read as complexity increases.

A cleaner approach is:

```
WITH DepartmentAverage AS
(
    SELECT DepartmentID,
           AVG(Salary) AS AvgSalary
    FROM Employees
    GROUP BY DepartmentID
)

SELECT *
FROM DepartmentAverage
WHERE AvgSalary > 50000;
```

This is called a:

Common Table Expression (CTE)



## Learning Objectives

After completing this module, you will be able to:

- ✓ Understand CTEs
- ✓ Create CTEs
- ✓ Use Multiple CTEs
- ✓ Understand Recursive CTEs
- ✓ Compare CTEs and Subqueries
- ✓ Compare CTEs and Temporary Tables
- ✓ Solve Interview Questions

---

## Table of Contents

---

1. What is a CTE?
2. Why Use CTEs?
3. CTE Syntax
4. Basic CTE Example
5. Multiple CTEs
6. CTE with JOINS
7. Recursive CTEs
8. CTE vs Subquery
9. CTE vs Temporary Table
10. Common Mistakes
11. Best Practices
12. Interview Questions
13. Business Analytics Examples
14. Summary
15. Practice Questions

---

## What is a CTE?

---

A CTE is a temporary named query result that can be referenced within a SQL statement.

---

## Key Characteristics

---

- ✓ Improves readability
- ✓ Simplifies complex queries
- ✓ Supports recursion

- ✔ Exists only during query execution

---

## Structure

---

```
WITH CTE_Name AS
(
    Query
)

SELECT *
FROM CTE_Name;
```

---

## Lifecycle

---

```
Query Starts
  ↓
CTE Created
  ↓
Query Executes
  ↓
CTE Disappears
```

---

## Why CTEs Matter

---

Used in:

- Reporting
- Data Analytics
- Dashboard Queries
- Recursive Hierarchies
- Interview Questions

---

## 2 Why Use CTEs?

---

Without CTE:

```
SELECT *
FROM
```

```
(  
    SELECT ...  
) A
```

---

With CTE:

```
WITH A AS  
(  
    SELECT ...  
)  
  
SELECT *  
FROM A;
```

---

Benefits

- ✓ Cleaner code
  - ✓ Easier debugging
  - ✓ Better maintainability
- 

## Example

---

Complex salary analysis becomes more readable.

---

### CTE Syntax

---

Basic syntax:

```
WITH CTE_Name AS  
(  
    SELECT Columns  
    FROM Table  
)  
  
SELECT *  
FROM CTE_Name;
```

---

## Important Rule

---

The CTE must be immediately followed by:

```
SELECT
INSERT
UPDATE
DELETE
MERGE
```

---

## Example

---

```
WITH EmployeesCTE AS
(
    SELECT *
    FROM Employees
)

SELECT *
FROM EmployeesCTE;
```



## Basic CTE Example

---

### Employees Table

| EmployeeID | Salary |
|------------|--------|
| 1          | 40000  |
| 2          | 60000  |
| 3          | 70000  |

---

### Query

```
WITH HighSalaryEmployees AS
(
    SELECT *
    FROM Employees
    WHERE Salary > 50000
)
```

```
SELECT *  
FROM HighSalaryEmployees;
```

Output

| EmployeeID | Salary |
|------------|--------|
| 2          | 60000  |
| 3          | 70000  |

## Why Useful?

Separates:

Filtering Logic

from

Final Query

## 5 Multiple CTEs

You can define multiple CTEs.

## Syntax

```
WITH CTE1 AS  
(  
    Query1  
)  
  
CTE2 AS  
(  
    Query2  
)  
  
SELECT *  
FROM CTE2;
```

# Example

---

```
WITH EmployeeCount AS
(
    SELECT DepartmentID,
           COUNT(*) AS TotalEmployees
    FROM Employees
    GROUP BY DepartmentID
),
DepartmentSalary AS
(
    SELECT DepartmentID,
           AVG(Salary) AS AvgSalary
    FROM Employees
    GROUP BY DepartmentID
)

SELECT *
FROM DepartmentSalary;
```

---

## Benefits

---

Organized logic.

---



## CTE with JOINS

---

Very common in analytics.

---

Example

```
WITH EmployeeDepartment AS
(
    SELECT E.EmployeeID,
           E.EmployeeName,
           D.DepartmentName

    FROM Employees E

    INNER JOIN Departments D
    ON E.DepartmentID = D.DepartmentID
)
```

```
SELECT *  
FROM EmployeeDepartment;
```

Output

Employee details with department names.

## Analytics Use

CTEs + JOINS are frequently used together.

## 7 Recursive CTEs

Advanced and important interview topic.

## What is Recursion?

A query references itself.

Used for:

- Employee Hierarchies
- Organization Trees
- Category Structures
- Folder Structures

### Example Table

| EmployeeID | EmployeeName | ManagerID |
|------------|--------------|-----------|
| 1          | CEO          | NULL      |
| 2          | Rahul        | 1         |
| 3          | Aman         | 2         |

## Recursive CTE Structure

```
WITH EmployeeHierarchy AS
(
    -- Anchor Query

    SELECT EmployeeID,
           EmployeeName,
           ManagerID

    FROM Employees

    WHERE ManagerID IS NULL

    UNION ALL

    -- Recursive Query

    SELECT E.EmployeeID,
           E.EmployeeName,
           E.ManagerID

    FROM Employees E

    INNER JOIN EmployeeHierarchy H

    ON E.ManagerID = H.EmployeeID
)

SELECT *
FROM EmployeeHierarchy;
```

---

## Components

---

### Anchor Member

Starting point.

CEO

---

### Recursive Member

Repeated execution.

Employees under CEO

---

# Result

---

Entire hierarchy generated.

---

## 8 CTE vs Subquery

---

Common interview question.

---

# Subquery

---

```
SELECT *
FROM
(
    SELECT ...
) A;
```

---

# CTE

---

```
WITH A AS
(
    SELECT ...
)

SELECT *
FROM A;
```

---

# Comparison

---

| Feature         | CTE    | Subquery |
|-----------------|--------|----------|
| Readability     | Better | Lower    |
| Recursion       | Yes    | No       |
| Reusability     | Better | Limited  |
| Complex Queries | Better | Harder   |

---

# Recommendation

For complex logic:

Use CTE

## CTE vs Temporary Table

Another common interview topic.

# CTE

Temporary Query Result

Exists only during execution.

# Temporary Table

Physical Temporary Object

Stored in tempdb.

# Comparison

| Feature     | CTE               | Temp Table |
|-------------|-------------------|------------|
| Storage     | No                | Yes        |
| Lifetime    | Single Query      | Session    |
| Performance | Small-Medium Data | Large Data |
| Recursion   | Yes               | No         |

# Use CTE When

Simple transformation  
Readable query  
Recursive logic

---

## Use Temp Table When

---

Large intermediate results  
Multiple references

---

## 10 Common Mistakes

---

---

### Missing Final Query

---

Bad:

```
WITH A AS  
(  
    SELECT *  
    FROM Employees  
)
```

Error.

CTE must be followed by query.

---

---

### Wrong Placement

---

Bad:

```
SELECT *  
FROM Employees  
  
WITH A AS (...)
```

---

Error.

---

## Infinite Recursion

---

Bad recursive logic.

Can exceed recursion limits.

---

## Naming Conflicts

---

Avoid duplicate CTE names.

---

## Best Practices

---

### Use Meaningful Names

Good:

```
HighSalaryEmployees  
DepartmentSummary  
MonthlySales
```

---

### Break Complex Queries

Use multiple CTEs.

---

### Use CTEs for Readability

Especially with JOINS.

---

### Avoid Excessive Nesting

Too many CTEs can become difficult.

---

### Use Recursive CTEs Carefully

Monitor performance.

---

# Professional Example

---

```
WITH DepartmentAverage AS
(
    SELECT DepartmentID,
           AVG(Salary) AS AvgSalary

    FROM Employees

    GROUP BY DepartmentID
)

SELECT *
FROM DepartmentAverage
WHERE AvgSalary > 50000;
```

---

## 1 2 Common Interview Questions

---

### What is a CTE?

Temporary named result set.

---

### Why Use CTEs?

Readability and maintainability.

---

### Can a CTE Reference Itself?

Yes.

Using recursion.

---

### Difference Between CTE and Subquery?

CTEs are easier to read and support recursion.

---

### Difference Between CTE and Temp Table?

Temp tables are physical.

CTEs are logical.

---

## Can Multiple CTEs Be Created?

Yes.

---

### 1 3 Business Analytics Examples

---

## High Salary Employees

---

```
WITH HighSalaryEmployees AS
(
    SELECT *
    FROM Employees
    WHERE Salary > 50000
)

SELECT *
FROM HighSalaryEmployees;
```

---

## Department Summary

---

```
WITH DepartmentSummary AS
(
    SELECT DepartmentID,
           COUNT(*) AS Employees,
           AVG(Salary) AS AvgSalary

    FROM Employees

    GROUP BY DepartmentID
)

SELECT *
FROM DepartmentSummary;
```

---

## Monthly Sales Analysis

---

```
WITH MonthlySales AS
(
```

```
SELECT MONTH(OrderDate) AS SalesMonth,  
       SUM(SalesAmount) AS Revenue  
  
FROM Sales  
  
GROUP BY MONTH(OrderDate)  
)  
  
SELECT *  
FROM MonthlySales;
```

---

## Customer Analytics

---

```
WITH CustomerOrders AS  
(  
    SELECT CustomerID,  
           COUNT(*) AS Orders  
  
    FROM Orders  
  
    GROUP BY CustomerID  
)  
  
SELECT *  
FROM CustomerOrders;
```

---

## Dashboard Layer

---

Many Power BI and Tableau data models use CTEs extensively.

---



## Summary

---

In this module, you learned:

- ✓ CTE Basics
- ✓ CTE Syntax
- ✓ Multiple CTEs
- ✓ CTE with JOINS
- ✓ Recursive CTEs

- ✓ CTE vs Subquery
  - ✓ CTE vs Temp Table
  - ✓ Analytics Use Cases
- 



## Practice Questions

---

### Theory

1. What is a CTE?
  2. Why use a CTE?
  3. What is the syntax of a CTE?
  4. Can multiple CTEs exist in one query?
  5. What is a Recursive CTE?
  6. Difference between CTE and Subquery?
  7. Difference between CTE and Temp Table?
  8. What is an Anchor Member?
  9. What is a Recursive Member?
  10. Why are CTEs useful in analytics?
- 

### Practical Exercises

#### Task 1

Create a simple CTE.

---

#### Task 2

Filter employees using a CTE.

---

#### Task 3

Create multiple CTEs.

---

#### Task 4

Join tables using a CTE.

---

#### Task 5

Build a department summary CTE.

---

#### Task 6

Create an employee hierarchy using a Recursive CTE.

---

## Challenge Project

Create:

```
Employees
Departments
Customers
Orders
Sales
```

Build:

- Employee Analytics
- Department KPIs
- Monthly Revenue Analysis
- Customer Segmentation
- Organization Hierarchy

using CTEs.

---



## Next Module

---

### → Module 24: Window Functions (Most Important Advanced SQL Topic)

Topics Covered:

- ROW\_NUMBER()
- RANK()
- DENSE\_RANK()
- NTILE()
- LEAD()
- LAG()
- FIRST\_VALUE()
- LAST\_VALUE()
- Running Totals
- Moving Averages
- Analytics Interview Questions